ТОО ”Колледж Хекслет”

Лабораторная работа №7

Разработка REST API и CRUD-операций

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Разработка REST API и CRUD-операций.

Цель: Разработать полноценный REST API для ресурса products, реализовать операции Create, Read, Update и Delete с использованием архитектуры routes → controllers → services и проверить полный CRUD-цикл через HTTP-клиент.

Условие: RO4_2/lesson7/ЛР-7.pdf

Требования задания

• использовать ресурсные URL;

• поддерживать GET списка товаров;

• поддерживать GET одного товара;

• поддерживать POST;

• поддерживать PATCH;

• поддерживать DELETE;

• возвращать 201 после создания товара;

• возвращать 404 для отсутствующего товара;

• использовать разделение Route → Controller → Service;

• не передавать req и res в Service;

• возвращать данные в JSON;

• успешно проходить полный CRUD-цикл.

Ход выполнения

Реализован полный цикл Create → Read → Update → Read → Delete → Read. PATCH сохраняет поля, не переданные в запросе. После удаления GET возвращает 404. Каждый ресурс имеет постоянный URL, а действие определяется HTTP-методом.

Проект: RO4_2/lesson7

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера: http://localhost:3000.

routes/productRoutes.js

routes/productRoutes.js

```
import { Router } from 'express';
import * as controller from '../controllers/productController.js';
import { validateId } from '../middleware/validateId.js';
import checkApiKey from '../middleware/checkApiKey.js';
const router = Router();
router.param('id', validateId);
router.use((req, res, next) => { console.log('Product Router:', req.method,
req.originalUrl); next(); });
router.get('/', controller.getProducts);
router.get('/:id', controller.getProductById);
router.post('/', checkApiKey, controller.createProduct);
router.patch('/:id', controller.updateProduct);
router.delete('/:id', controller.deleteProduct);
router.patch('/:id/settings', controller.productSettings);
export default router;
```

Листинг 1. routes/productRoutes.js

controllers/productController.js

controllers/productController.js

```
import * as service from '../services/productService.js';
import { AppError } from '../errors/AppError.js';

export async function getProducts(req, res, next) {
    try { res.json(await service.getAllProducts(req.query)); }
    catch (error) { next(error); }
}
export async function getProductById(req, res, next) {
    try {
        const product = await service.findProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function createProduct(req, res, next) {
    try { res.status(201).json(await service.addProduct(req.body)); }
    catch (error) { next(error); }
}
export async function updateProduct(req, res, next) {
    try {
        const product = await service.updateProductById(Number(req.params.id),
req.body);
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export async function deleteProduct(req, res, next) {
    try {
        const product = await service.deleteProductById(Number(req.params.id));
        if (!product) return next(new AppError('Product not found', 404));
        res.json(product);
    } catch (error) { next(error); }
}
export function productSettings(req, res) {
    res.json({ id: Number(req.params.id), notify: req.query.notify === 'true', changes:
```

Листинг 2. controllers/productController.js (часть 1)

controllers/productController.js

```
req.body });
}
```

Листинг 3. controllers/productController.js (часть 2)

services/productService.js

services/productService.js

```
let products = [
    {
        "id": 1,
        "name": "Keyboard",
        "price": 15000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 2,
        "name": "Mouse",
        "price": 8000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 3,
        "name": "Monitor",
        "price": 85000,
        "category": "Monitors",
        "inStock": false
    },
    {
        "id": 4,
        "name": "Laptop",
        "price": 420000,
        "category": "Computers",
        "inStock": true
    },
    {
        "id": 5,
        "name": "Webcam",
        "price": 22000,
        "category": "Accessories",
        "inStock": false
```

Листинг 4. services/productService.js (часть 1)

services/productService.js

```
    }
];
let nextId = 6;

export function getAllProducts(query = {}) {
    return products.filter(product =>
        (query.category === undefined || product.category === query.category) &&
        (query.inStock === undefined || product.inStock === (query.inStock === 'true'))
&&
        (query.minPrice === undefined || product.price >= Number(query.minPrice)) &&
        (query.maxPrice === undefined || product.price <= Number(query.maxPrice))
    );
}
export function findProductById(id) {
    return products.find(product => product.id === id);
}
export function addProduct(data) {
    const product = { ...data, id: nextId++ };
    products.push(product);
    return product;
}
export function updateProductById(id, changes) {
    const product = findProductById(id);
    if (!product) return null;
    Object.assign(product, changes);
    return product;
}
export function deleteProductById(id) {
    const index = products.findIndex(product => product.id === id);
    if (index === -1) return null;
    return products.splice(index, 1)[0];
}
```

Листинг 5. services/productService.js (часть 2)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

• Полный CRUD-цикл

• Совместная фильтрация

• Уникальные id после удаления

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

GET /api/products/3 → 200

GET /api/products/999 → 404

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?inStock=false → 200

POST /api/products → 403

POST /api/products → 403

POST /api/products → 201

GET /api/products/6 → 200

PATCH /api/products/6 → 200

PATCH /api/products/4/settings?notify=true → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

PATCH /api/products/6 → 404

DELETE /api/products/6 → 404

POST /api/products → 201

GET /api/unknown → 404

POST /api/orders → 201

GET /api/orders/1 → 200

PATCH /api/orders/1 → 200

GET /api/orders/999 → 404

Примеры ответов

POST /api/products → 201


```
{
  "name": "SSD 2TB",
  "price": 65000,
  "category": "Components",
  "inStock": true,
  "id": 6
}
```


PATCH /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


DELETE /api/products/6 → 200


```
{
  "name": "SSD 2TB",
  "price": 42000,
  "category": "Components",
  "inStock": false,
  "id": 6
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.